

Selected Subjects in Robotics and AI HW2

Daniel Haim Breger, 316136944

Ethan Chelly, 346395064

Technion

(Date: January 30, 2026)

1 Fourier-Based Signal Processing

(a) Analytical Derivation of $X(f)$

Consider the continuous-time signal $x(t) = e^{-a|t|} \cos(2\pi f_c t)$ with $a > 0$. To derive the Fourier Transform $X(f)$, we utilize the linearity and modulation properties of the Continuous Fourier Transform (CFT). **Step 1: Transform of the two-sided exponential**

Let $g(t) = e^{-a|t|}$. By definition of the CFT:

$$G(f) = \int_{-\infty}^{\infty} e^{-a|t|} e^{-j2\pi f t} dt = \int_{-\infty}^0 e^{(a-j2\pi f)t} dt + \int_0^{\infty} e^{-(a+j2\pi f)t} dt$$

Evaluating the improper integrals:

$$G(f) = \left[\frac{e^{(a-j2\pi f)t}}{a-j2\pi f} \right]_{-\infty}^0 + \left[\frac{e^{-(a+j2\pi f)t}}{-(a+j2\pi f)} \right]_0^{\infty} = \frac{1}{a-j2\pi f} + \frac{1}{a+j2\pi f}$$

Combining the terms leads to the Lorentzian spectrum:

$$G(f) = \frac{(a+j2\pi f) + (a-j2\pi f)}{a^2 + (2\pi f)^2} = \frac{2a}{a^2 + 4\pi^2 f^2}$$

Step 2: Modulation Property

The signal $x(t) = g(t) \cos(2\pi f_c t)$ can be expressed using Euler's identity. The modulation property states that multiplying by $\cos(2\pi f_c t)$ shifts the spectrum to $\pm f_c$:

$$X(f) = \frac{1}{2} [G(f - f_c) + G(f + f_c)]$$

Substituting $G(f)$:

$$X(f) = \frac{a}{a^2 + 4\pi^2(f - f_c)^2} + \frac{a}{a^2 + 4\pi^2(f + f_c)^2}$$

The magnitude spectrum $|X(f)|$ consists of two symmetric Lorentzian peaks centered at $\pm f_c$.

(b) Mechanism of Signal-Noise Separation in the Frequency Domain

The Fourier Transform is a fundamental tool for enhancing the Signal-to-Noise Ratio (SNR) by exploiting the spectral distribution differences between deterministic signals and stochastic noise.

- **Spectral Energy Density:** In the time domain, a signal $x(t)$ and additive noise $n(t)$ are often inextricably mixed. However, the Fourier Transform $X(f)$ of the exponential-modulated signal derived in part (a) shows that the signal energy is highly concentrated in narrow Lorentzian peaks centered at $\pm f_c$. In contrast, common environmental noise (such as AWGN) tends to be broadband, spreading its power spectral density across a wide range of frequencies.
- **Filtering as a Masking Operation:** By defining a Band-Pass Filter (BPF) with a transfer function $H(f)$, we can isolate the signal. If we set $H(f) \approx 1$ for $f \in [f_c - \Delta f, f_c + \Delta f]$ and $H(f) \approx 0$ elsewhere, the output $Y(f) = [X(f) + N(f)]H(f)$ retains the majority of the signal energy while discarding the noise power present in the rest of the spectrum.
- **Engineering Trade-off:** The parameter a in $e^{-a|t|}$ dictates the bandwidth of the Lorentzian peaks.

A smaller a (slower decay) results in a narrower spectral footprint, allowing for a tighter filter and superior noise rejection, albeit at the cost of transient response speed.

(c) Analytical Treatment of Aliasing

Aliasing is the phenomenon where high-frequency components are "folded" into lower frequencies during the sampling process, creating irreversible distortion. **Mathematical Derivation:** Sampling a continuous signal $x(t)$ at a frequency $f_s = 1/T_s$ is mathematically equivalent to multiplying $x(t)$ by an impulse train $\sum_{k=-\infty}^{\infty} \delta(t - kT_s)$. According to the multiplication property of the Fourier Transform, this corresponds to a convolution in the frequency domain:

$$X_s(f) = X(f) * \mathcal{F} \left\{ \sum_{k=-\infty}^{\infty} \delta(t - kT_s) \right\} = X(f) * \left(f_s \sum_{k=-\infty}^{\infty} \delta(f - kf_s) \right)$$

This yields the periodic summation:

$$X_s(f) = f_s \sum_{k=-\infty}^{\infty} X(f - kf_s)$$

Condition for Distortion ($f_s < 2f_{max}$): The Nyquist-Shannon sampling theorem states that to recover $x(t)$, we must have $f_s \geq 2f_{max}$. If $f_s < 2f_{max}$, the shifted spectra $X(f - kf_s)$ overlap. Specifically, for our signal $x(t) = e^{-a|t|} \cos(2\pi f_c t)$, which is theoretically not band-limited, the tails of the Lorentzian peaks at $k = 0$ and $k = 1$ will overlap regardless of f_s . However, practically, if f_s is low, the magnitude of $X(f - f_s)$ in the baseband $[-f_s/2, f_s/2]$ becomes significant, making it impossible to reconstruct the original signal without interference from the periodic aliases.

(d) Practical Consequences and Mitigation Strategies

Consequences in Engineering Systems:

- **Robotics and Control:** In high-speed robotic joints, aliasing in encoder or IMU data can transform high-frequency motor vibrations into low-frequency "phantom" oscillations. This can lead to instability in PID controllers or erroneous state estimation in Kalman filters.
- **Image Processing:** Aliasing manifests as Moiré patterns (interference fringes) or "jaggies" on edges. In medical imaging or satellite surveillance, these artifacts can lead to critical misinterpretations of structural data.

Prevention Methods:

1. **Analog Anti-Aliasing Filter (AAF):** To adhere to the sampling theorem, a physical Low-Pass Filter (LPF) must be placed *before* the Analog-to-Digital Converter (ADC). This filter must have a cut-off frequency $f_{cut} \leq f_s/2$ to attenuate any frequency components that would otherwise fold back into the baseband.
2. **Oversampling and Digital Decimation:** By sampling at a much higher rate ($f_{s,high} \gg 2f_{max}$), we create a wide "guard band" between spectral copies. This relaxes the requirements on the analog AAF (allowing for a gentler rolloff) and permits the use of sharp, high-order digital filters to remove noise before downsampling to the target rate.

2 Convolutional Neural Networks (CNNs)

(a) Functional Role and Architectural Justification

The proposed architecture follows the classic hierarchical pattern of feature extraction followed by global classification.

- **Convolutional Layer (16 filters, 3×3):** This layer serves as the primary feature extractor. By utilizing a local receptive field, the layer captures spatial correlations (edges, textures) that are lost in standard dense networks. The use of 16 filters allows the network to learn multiple distinct feature maps in parallel. The "weight sharing" property is critical here, as the same 3×3 kernel is applied across the entire image, drastically reducing the parameter count compared to a fully connected layer.
- **ReLU Activation:** The Rectified Linear Unit, defined as $f(x) = \max(0, x)$, introduces the necessary non-linearity. Its primary advantage in this architecture is the mitigation of the vanishing gradient problem, allowing for more efficient backpropagation by maintaining a constant gradient for all positive activations.
- **Max-Pooling (2×2):** This downsampling operation reduces the spatial resolution of the feature maps. By selecting the maximum value in each 2×2 window, the layer provides a degree of *translation invariance*, meaning the network can recognize a feature even if its position in the frame shifts slightly. It also serves to prevent overfitting by reducing the total number of parameters in the subsequent dense layers.
- **Fully Connected (FC) Layer (100 neurons):** After local features are extracted and downsampled, this layer performs "high-level" reasoning. It integrates information from all 15,376 spatial features to form a global representation of the input image.
- **Softmax Output (10 classes):** The final layer maps the hidden representations to a 10-dimensional vector. The Softmax function normalizes these scores into probabilities ($p_i \in [0, 1]$ and $\sum p_i = 1$), which is essential for multi-class classification.

(b) Detailed Spatial Dimension Analysis

To track the transformation of the data volume, we apply the general formula for output dimensions: $O = \frac{I-K+2P}{S} + 1$, where I is input size, K is kernel size, P is padding, and S is stride.

1. **Input Volume:** $V_0 = 64 \times 64 \times 3$ (RGB channels).
2. **Convolutional Layer:** Given $K = 3, S = 1, P = 0$:

$$O_{conv} = \frac{64 - 3 + 0}{1} + 1 = 62$$

The resulting volume is $62 \times 62 \times 16$, where the depth reflects the 16 filters.

3. **ReLU Activation:** Point-wise operation; dimensions remain $62 \times 62 \times 16$.
4. **Max-Pooling Layer:** Given $K = 2, S = 2$:

$$O_{pool} = \frac{62 - 2}{2} + 1 = 31$$

The resulting volume is $31 \times 31 \times 16$.

5. **Flattening:** Before the FC layer, the 3D volume is serialized into a 1D vector:

$$D_{flatten} = 31 \times 31 \times 16 = \mathbf{15,376} \text{ units.}$$

(c) Trainable Parameter Derivation

The total number of parameters is the sum of weights ($\mathbf{W}$) and biases ($\mathbf{b}$) for each layer. **1. Convolutional Layer:** Each of the 16 filters has a volume of $3 \times 3 \times 3$ (width $\times$ height $\times$ input depth).

- Weights: $(3 \times 3 \times 3) \times 16 = 432$
- Biases: 16 (one per filter)
- **Layer Total:** $432 + 16 = \mathbf{448}$.

2. First Fully Connected Layer: Every neuron in the 100-unit layer is connected to every element of the 15,376-unit input vector.

- Weights: $15,376 \times 100 = 1,537,600$
- Biases: 100
- **Layer Total:** $1,537,600 + 100 = \mathbf{1,537,700}$.

3. Output Softmax Layer: Maps 100 hidden neurons to 10 class scores.

- Weights: $100 \times 10 = 1,000$
- Biases: 10
- **Layer Total:** $1,000 + 10 = \mathbf{1,010}$.

Total Network Parameters:

$$P_{total} = 448 + 1,537,700 + 1,010 = \mathbf{1,539,158}$$

Note: Approximately 99.9% of the network's parameters are contained within the first fully connected layer, highlighting the memory-intensive nature of transitioning from convolutional feature maps to dense classification layers.

3 Anomaly Detection in Signals

3.1 Classical Statistical Methods For Anomaly Detection In Time Series Signals

We'll break this question down into the different methods mentioned and explain how each one can be used for anomaly detection. All of the methods assume there is an expected behavior and the anomalies deviate from the expected behavior.

3.1.1 Thresholding

If we assume the signal has some median value and a certain distribution around it, we can set an upper bound on the deviation from the median that's considered acceptable. Values exceeding this threshold will be flagged as anomalies. This is very simple to implement but its downsides are obvious: the assumption that the signal is well behaved around some median and doesn't spike too often.

3.1.2 Spectral Analysis

If we know our signal is expected to be comprised of a specific set or range of frequencies, we can perform a fourier transform on it and in the frequency domain we will have an acceptable range or set of frequencies, with any frequency outside this range or set being considered anomalous. This method is useful for signals which either are expected to be periodic, carried by specific frequencies, or are specifically characterized by their frequency. The downsides to this method is the method by which the signal is analyzed itself may introduce artifacts or lose information, non-repeating anomalies may be hard to detect, very slow changes in the signal may be hard to detect, and of course if the signal isn't strongly characterized by its frequency this method won't be effective.

3.1.3 Kalman Filtering Residuals

Kalman filtering gives an optimal linear estimator for a signal. By assuming a model for the signal, we can use a Kalman filter to predict the next value of the signal. By monitoring the difference between the predicted value and the measured value and setting a threshold on this difference, we can declare measurements which exceed this threshold (which may be dynamic or static) as anomalies. This method is useful when we have a good understanding of the system generating this data and can model it well, and the system in question behaves in a linear manner. The downsides to Kalman Filtering are that it assumes the system is linear in its evolution, and that we have a good model of the system.

3.2 Encoder-Decoder For Anomaly Detection

If we treat an incoming signal as a d_1 dimensional vector but we know the underlying signal is of a lower dimensionality, we can train a neural network to compress this vector into a $d_2 < d_1$ dimensional vector, then expand it back to a d_1 dimensional vector. We can then train this network on known good data, and the NN will learn to reconstruct the signal. Anomalies in the signal are much more likely to live in the subspace orthogonal to the signal's subspace (due to the dimensionality increase). If we look at the difference between the measured signal and the reconstructed signal and call that the error, a high error implies that either there's a lot of noise (aka anomalies), or the signal we tried to compress dimensionally was actually of higher dimension than we expected. We could also use train the NN to instead forecast the next value of the signal, rather than reconstruct the last received signal, which operates similarly, the

main difference being that it is more sensitive to sharp sudden anomalies which are not continuous over the signal. The downsides are that if the lower dimension of the NN is incorrectly selected, it can either fail due to it missing dimensionality in the data or in the opposite direction fail due to it encoding anomalies as signal in redundant dimensions. In addition, since it requires training a neural network, you need a large amount of data to train the model beforehand.

3.3 Classical vs deep learning

The main, obvious, difference is in the philosophy of the approach. Classical methods assume specific models for the signal and utilize statistical methods to analyze the behavior of the signal. Deep learning methods instead are based on analyzing patterns in existing data to learn the signal's behavior.

3.3.1 Classical Methods

Classical methods' strengths are:

1. they are based on mathematical models, which make them understandable and explainable.
2. Do not require large datasets to perform well.
3. Can be designed specifically for the signal in question using knowledge about the signal.
4. Much lighter computationally and can be used for performance sensitive tasks.

Classical methods' weaknesses are:

1. Most classical methods assume a linear system, so any non-linear behavior is a challenge.
2. The effectiveness of the system is completely dependant on how well the engineer designing the system understands the signal and possible anomalies.

3.3.2 Encoder-Decoder

Encoder-Decoder methods' strengths are:

1. Are able to learn more complex behavior both in terms of the signal and the anomalies, which humans may not even know exists.
2. Works well in systems with large amounts of data.
3. Requires minimal prior knowledge of the signal or anomalies.
4. Can model nonlinear systems.
5. Can be easily scaled.

Encoder-Decoder methods' weaknesses are:

1. Risk of overfitting.
2. Susceptible to learning incorrect behavior from the training data if not properly trained.
3. If data is limited, performs very poorly.
4. Impossible to explain how the NN works, making unexpected behavior hard to diagnose.
5. Training is expensive computationally.

4 Visual Object Detection Using CNNs

The code for this question is provided in the file q4.ipynb. We chose to work with the Oxford Pets dataset. Originally we wanted to work with VOC2012 but found it far too challenging.

4.1 Data preprocessing

Each image is resized to 224×224 pixels with the bounding boxes transformed accordingly, then some basic augmentation is applied (simple color jitter and horizontal flips).

4.2 CNN Architecture

To answer the question we made two CNNs. The first was defined with a set architecture which was trained and evaluated with a specific (guessed) hyperparameter set, then another CNN was defined which could be modified for the comparisons.

4.2.1 First CNN

The first CNN's architecture is based on a backbone which splits into a detection head and a classification head. The backbone is built of four blocks, each performing two 3×3 convolution layers, a batch norm, and 2×2 max pooling. The number of channels is gradually increased from 3 to 256. The classification head then consists of a linear layer, a ReLU, and another linear layer. The identification head consists of two sets of linear and relu layers, then a final linear layer to compress to 4 dimensions of prediction.

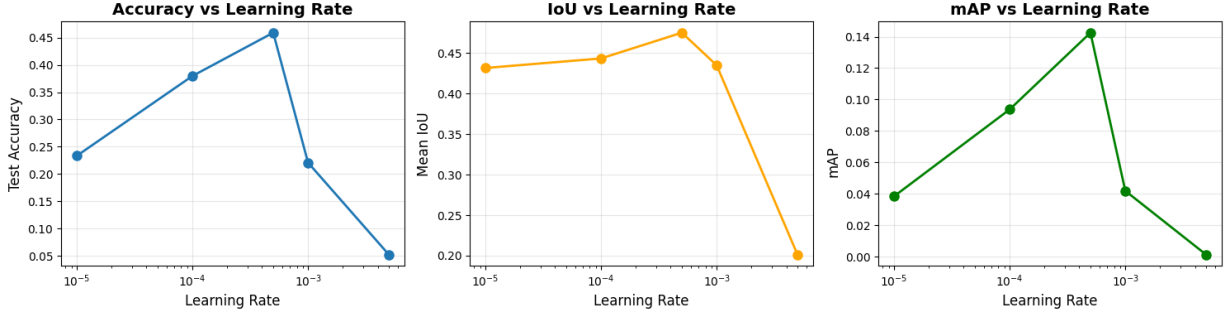
After training for 10 epochs as a test to see the architecture makes any sense with a learning rate of $1e-3$ and bias towards the bounding box loss (as we noticed that the bounding box is much slower to move) we saw it indeed improved on random guesses so we considered it good enough to move on to the hyperparameter analysis.

4.2.2 Flexible CNN

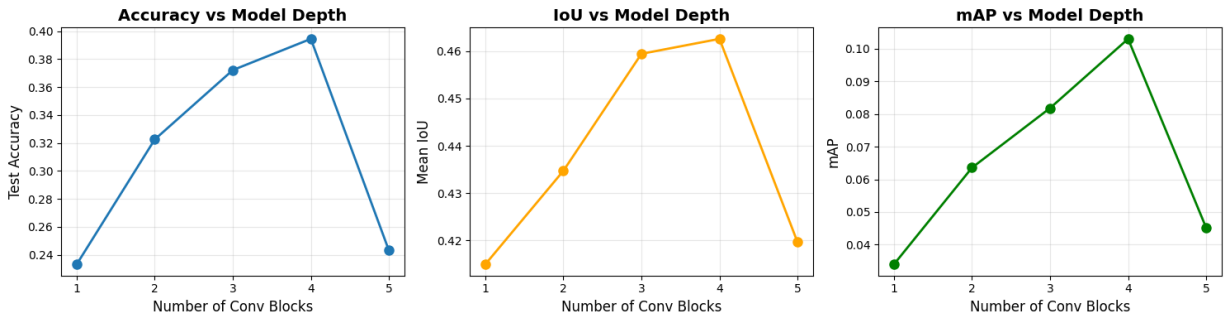
The second CNN is a version of the first architecture designed with changable hyperparameters. It maintains the same VGG-inspired dual-head design but can be changed significantly in terms of the inner architecture. The number of convolutional layers can be changed by increased or decreasing the number of predefined blocks used in the network, and the filter size of the convolutions can be changed too. The blocks composing the backbone are made of two convolution layers with a batch norm and relu after each convolution, 2×2 and max pooling for downsampling. The classification and identification heads remain identical to the simple CNN and are applied after the backbone.

4.2.3 Results

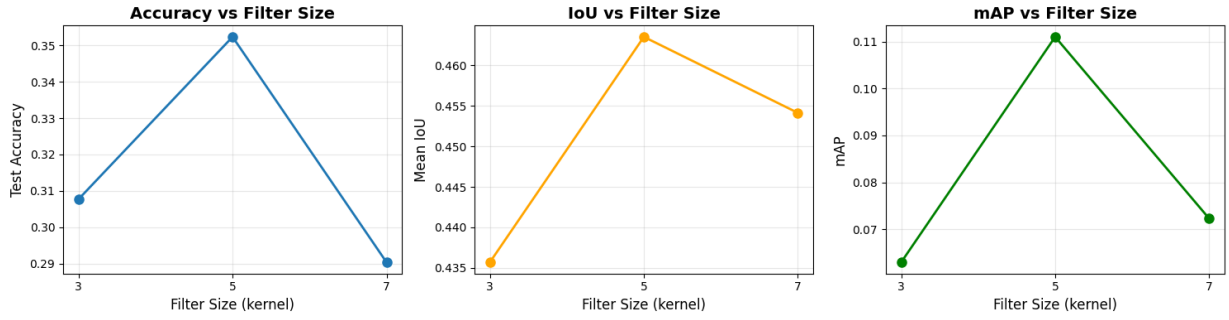
The first test conducted was the learning rate comparison. In this test we defined the network with default parameters (resulting in an identical architecture to the first CNN) and ran the test for 50 epochs with learning rates ranging from 1×10^{-5} to 5×10^{-3} . It is quite easy to see that the best learning rate is



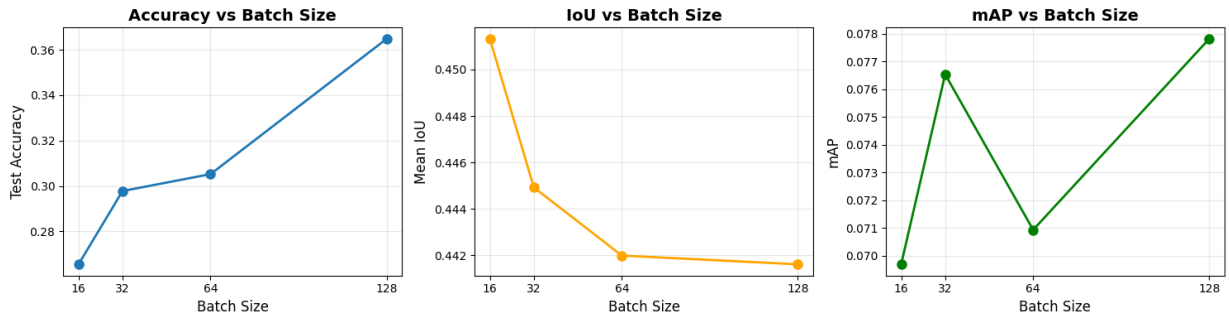
5×10^{-4} . The accuracy achieved is 0.4591, with an IoU of 0.475 and mAP of 0.1425. These numbers are all of course not great, but given the custom architecture and short training we expected as much. From here all the tests were performed with a learning rate of 5×10^{-4} and for 50 epochs each. The next test performed was the test comparing different numbers of convolutional layers. In this test we varied the number of convolution blocks between 1 and 5. As can be seen from the figure, the optimal depth for all



metrics is achieved at 4 blocks, at which point the accuracy is 0.3945, IoU is 0.4626, and mAP is 0.1031. Interestingly, the increase from 4 blocks to 5 completely destroys performance making it almost as poor as a single convolutional block. This likely implies that at 5 blocks the network begins extracting noise from the images and classifying it. From here the following tests used 4 convolution blocks. The next test we performed was to vary the filter sizes of the convolutions. We varied the size to be either 3×3 , 5×5 , or 7×7 .



As can be clearly seen, the 5×5 kernel size performs the best across all metrics. This indicates that the spatial features in the images in the dataset have a certain level of feature complexity which is captured best by this specific size. Too small and the convolutions become too localized, too large and fine details get drowned out. For the final comparison we therefore used 5×5 kernels. The final test we performed was to vary the batch size. We tested batch sizes of 16, 32, 64, and 128. Unlike the previous tests, here we see a



clear divergence in optimality between the different metrics. If optimizing for accuracy, the larger the batch size the more accurate the model becomes. This makes sense because larger batches means the model sees more elements at the same training round, meaning it is more likely to learn more general features rather than overfit in each batch. The IoU on the other hand drops the larger the batch is. We believe the reason for this is that the bounding boxes are distributed fairly uniformly within the image across the samples, and as the batch size increases, the variation in the locations becomes too great for the model to learn and instead it learns to either average it out or guess a large box that covers all the options better than failing for most of the boxes but fitting few very well. As for the mAP, the decrease in IoU is balanced out by the Accuracy, as the increase towards 128 is significant while the decrease from 64 to 128 in IoU is small (Which we believe is because it is approaching the asymptotical minimum for a random guess).

4.2.4 Discussion

As for the architecture used, we believe the double-head for detection and classification justified itself, with performance being unimpressive but effective and balanced between the two tasks. We introduced weighted loss, which let us counteract the tendency of the network to focus on optimizing the classification task at the expense of the identification task, and the VGG-inspired architecture did capture spatial features.

As for the hyperparameters, we found clear optimal values for most hyperparameters with the exception of batch size, which inherently has opposite effects on the different metrics used for our goals. The architecture and training parameters could be significantly improved of course.